VPS Tools - Complete Usage Guide

Comprehensive documentation for all VPS Tools scripts. Each script can be run independently or as part of the automated cron schedule.

Table of Contents

1. Provisioning
2. Monitoring & Health
3. Security
4. Docker Management
5. Maintenance
6. Orchestration
7. Configuration
8. Troubleshooting

Provisioning

vps-build.sh

Complete VPS setup and configuration. Interactive script that provisions fresh Ubuntu 24.04 systems or reconfigures existing ones.

Usage:

```bash
sudo bash vps-build.sh
```

Modes:

1. **Fresh Install -** Complete provisioning
    - System updates (apt upgrade, dist-upgrade)
    - Hostname and timezone configuration
    - SSH hardening and key-based authentication
    - Non-root user creation
    - GitHub SSH key integration

- UFW firewall with app-specific rules
- Swap configuration
- Unattended security updates
- System hardening (sysctl parameters)
- Optional application installation

2. **Reconfigure -** Adjust specific settings
    - SSH keys from GitHub (add/update)
    - SSH configuration (port, root login, password auth)
    - Firewall rules (add, delete, reset)
    - Hostname changes
    - Timezone changes
3. **Troubleshoot -** System status review and adjustment
    - Full configuration summary
    - Interactive adjustment menu
    - Service status checking

Interactive Prompts:

```
Hostname: [auto-generated default]
Timezone: UTC (e.g., Europe/Dublin)
SSH Port: 22
Block root SSH login: yes (prohibit-password - Coolify compatible)
Create non-root user: yes
Username: ubuntu
Add GitHub SSH keys: yes/no
Disable password auth: yes
Enable UFW: yes
Create swap: yes (size in GB)
Unattended updates: yes
Application: docker|coolify|dokploy|n8n|mailu|nextcloud|zentyal|none
```

Application Details:

- **Docker + Portainer**: Container runtime with web UI (port 9000)
- **Coolify**: Self-hosted PaaS (ports 80, 443, 3000, 8000)
- **Dokploy**: Container deployment (port 3000)
- **n8n**: Workflow automation (port 5678)
- **Mailu**: Mail server (SMTP, IMAP, POP3 + web)

- **Nextcloud**: File sync and sharing (port 80)
- **Zentyal**: Network infrastructure (port 443)

SSH Security:

- `prohibit-password` - Allows key-based auth only (default, Coolify compatible)
- `yes` - Allows all authentication methods
- `no` - Blocks root access entirely

Monitoring & Health

vps-health-monitor.sh

Real-time system health monitoring with configurable alerts.

Usage:

```bash
sudo bash vps-health-monitor.sh [--check=resource] [--alert-email=email]
```

Checks Performed:

Check	Thresholds	Description
Disk	80% warn, critical	Root & mounted filesystem usage
Memory	75% warn, 85% critical	RAM usage with available tracking
CPU Load	>80% CPU warn, >100% critical	System load average
Services	SSH, UFW, Docker, unattended-upgrades	Critical service status
Docker	Container count, running status	Containers and images
SSH Config	Port, root login, password auth	SSH security settings
Firewall	UFW status and rules	Active firewall configuration
Failed Logins	24 hours, threshold=10	SSH authentication failures
SSL Certificates	Expiration tracking	Let's Encrypt/self-signed certs
Updates	Security/regular packages	Available system updates
Swap	Total and usage	Swap configuration status

Examples:

```bash
# Full health check
sudo bash vps-health-monitor.sh

# Check specific resource
sudo bash vps-health-monitor.sh --check=docker

# Enable email alerts
sudo bash vps-health-monitor.sh --alert-email=admin@example.com
```

Output Colors:

- 🟢 Green: Healthy/normal
- 🟡 Yellow: Warning threshold reached
- 🔴 Red: Critical threshold exceeded

---

vps-service-monitor.sh

Monitor systemd services and Docker containers with auto-restart capability.

Usage:

```bash
sudo bash vps-service-monitor.sh [--dry-run] [--alert-email=email]
```

Configuration:

Default config at `/etc/vps-tools/service-monitor.conf`:

```bash
# Service monitoring: "service:type:max_restarts:restart_delay"
SERVICES=(
  "ssh:systemd:0:60"        # SSH, no restart limit, check every 60s
  "ufw:systemd:0:60"        # UFW, no restart limit
  "docker:systemd:3:120"    # Docker, max 3 restarts/hour, check every 120s
)

# Docker containers: "name:max_restarts:restart_delay"
DOCKER_CONTAINERS=(
  "coolify:3:120"
  "dokploy:3:120"
  "n8n:3:120"
)
```

Features:

- Service status checking (systemd, Docker)
- Automatic restart on failure
- Rate limiting to prevent restart loops
- Hourly restart counters
- Container health status
- Email alerts on restart

Examples:

```bash
```

```bash
# Check service status
sudo bash vps-service-monitor.sh

# Test without making changes
sudo bash vps-service-monitor.sh --dry-run

# Enable alerts
sudo bash vps-service-monitor.sh --alert-email=admin@example.com
```

---

vps-log-analyzer.sh

Parse system and application logs for errors and warnings.

Usage:

```bash
sudo bash vps-log-analyzer.sh [--days=7] [--type=auth|service|system|all] [--alert-email=email]
```

Log Types:

Type	Analysis
auth	Failed logins, invalid users, successful connections
service	SSH, Docker, UFW, unattended-upgrades logs
system	Kernel errors, OOM events, disk I/O, segfaults
all	Complete analysis (default)

Examples:

```bash
```

```bash
# Analyze last 7 days
sudo bash vps-log-analyzer.sh --days=7

# Auth failures only
sudo bash vps-log-analyzer.sh --type=auth --days=14

# System errors only
sudo bash vps-log-analyzer.sh --type=system --days=30

# Send report via email
sudo bash vps-log-analyzer.sh --alert-email=admin@example.com
```

---

vps-backup-verifier.sh

Monitor backup status and verify backup integrity.

Usage:

```bash
sudo bash vps-backup-verifier.sh [--verify-all] [--alert-email=email]
```

Configuration:

Default config at `/etc/vps-tools/backup-verifier.conf`:

```bash
# Backup locations: "path:type:retention:max_age"
BACKUP_LOCATIONS=(
    "/opt/backups:archive:720:168"      # Keep 30 days, alert if >7 days old
    "/var/lib/docker/volumes:docker:720:168"
)

# Docker volumes to backup
DOCKER_VOLUMES=(
    "portainer_data:/opt/backups/portainer"
    "coolify:/opt/backups/coolify"
)

# Database backups
DATABASES=(
    "postgresql:postgres:/opt/backups/postgres"
    "mysql:mysql:/opt/backups/mysql"
)
```

Features:

- Backup age monitoring
- File count and size tracking
- Checksum verification
- Database accessibility checks
- Docker volume backup status
- Retention policy validation

Examples:

```bash
# Check backup status
sudo bash vps-backup-verifier.sh

# Verify checksums
sudo bash vps-backup-verifier.sh --verify-all

# Email alerts
sudo bash vps-backup-verifier.sh --alert-email=admin@example.com
```

Security

vps-ssh-audit.sh

SSH key management and security auditing.

Usage:

```bash
sudo bash vps-ssh-audit.sh [--audit|--rotate-user=username] [--alert-email=email]
```

Audit Checks:

- Key type validation (Ed25519 > ECDSA > RSA > DSS)
- Weak key detection and alerts
- Duplicate key identification
- GitHub key status verification

- Key age and rotation recommendations
- File permission validation

Examples:

```bash
# Audit all SSH keys
sudo bash vps-ssh-audit.sh --audit

# Rotate keys for specific user
sudo bash vps-ssh-audit.sh --rotate-user=ubuntu

# Check GitHub keys
sudo bash vps-ssh-audit.sh --audit | grep "GitHub"

# Send alert if issues found
sudo bash vps-ssh-audit.sh --audit --alert-email=admin@example.com
```

Key Strength Recommendations:

- ✅ Ed25519: Preferred (256-bit)
- ✅ ECDSA: Good (256-bit)
- ⚠️ RSA: Acceptable (4096-bit minimum)
- ❌ DSS: Weak (FIPS 186-4 deprecated)

---

vps-failed-login-reporter.sh

Analyze SSH authentication failures and detect attacks.

Usage:

```bash
sudo bash vps-failed-login-reporter.sh [--days=7] [--threshold=10] [--block-ips] [--alert-email=email]
```

Features:

- Failed password attempt tracking
- Invalid user detection
- Authentication method analysis (publickey vs password)
- Brute force pattern detection

- Rate limiting verification
- Geographic distribution (requires geoiplookup)
- IP blocking (automatic or manual)

Analysis:

```bash
# Failed passwords: 25+ alerts
Failed password: 10+ attempts in threshold
Invalid users: 5+ attempts

# Brute force detection: 5+ failures/hour from single IP
Attack pattern: High frequency login attempts
```

Examples:

```bash
# Analyze past week
sudo bash vps-failed-login-reporter.sh --days=7

# Check with custom threshold
sudo bash vps-failed-login-reporter.sh --threshold=5

# Block suspicious IPs automatically
sudo bash vps-failed-login-reporter.sh --block-ips

# Detailed report with geography
sudo bash vps-failed-login-reporter.sh --days=14 --alert-email=admin@example.com
```

---

vps-ssl-checker.sh

Monitor SSL/TLS certificate expiration across all platforms.

Usage:

```bash
sudo bash vps-ssl-checker.sh [--warn-days=30] [--alert-email=email]
```

Checks:

- Let's Encrypt certificates

- Docker service certificates
- Nginx/Apache certificates
- Self-signed certificate detection
- Certificate chain validation
- SHA256 fingerprints
- Expiration tracking

Thresholds:

- ❌ Expired: Certificate past expiration date
- 🔴 Critical: <7 days until expiration
- 🟡 Warning: <30 days (default, configurable)
- 🟢 OK: >30 days remaining

Examples:

```bash
# Check all certificates
sudo bash vps-ssl-checker.sh

# Custom warning threshold (60 days)
sudo bash vps-ssl-checker.sh --warn-days=60

# Email if issues found
sudo bash vps-ssl-checker.sh --alert-email=admin@example.com
```

---

vps-open-ports-auditor.sh

Scan open ports and validate against expected configuration.

Usage:

```bash
sudo bash vps-open-ports-auditor.sh [--expected-ports=22,80,443] [--scan-external] [--alert-email=email]
```

Features:

- Listening port enumeration
- Process-to-port mapping

- UDP port detection
- UFW firewall rule review
- Expected vs unexpected port validation
- External port scan (requires nmap)
- Common service port checking
- High-numbered port detection (potential backdoors)

Examples:

```bash
# Scan all open ports
sudo bash vps-open-ports-auditor.sh

# Validate against expected ports
sudo bash vps-open-ports-auditor.sh --expected-ports=22,80,443,3000

# External scan
sudo bash vps-open-ports-auditor.sh --scan-external

# Alert on unexpected ports
sudo bash vps-open-ports-auditor.sh --expected-ports=22,80,443 --alert-email=admin@example.com
```

Docker Management

vps-docker-health.sh

Real-time container health monitoring dashboard.

Usage:

```bash
sudo bash vps-docker-health.sh [--interval=60] [--restart-unhealthy] [--alert-email=email]
```

Dashboard Displays:

- Container status (up, exited, unhealthy, starting)
- CPU and memory usage per container
- Container health status (healthy/unhealthy/none)
- Recent error logs (last hour)

- Volume status and usage
- Image status (dangling, unused)
- Network configuration
- Service restart counts

Features:

- Auto-refresh at configurable intervals
- Color-coded health status
- CPU usage warnings (>50% yellow, >75% red)
- Automatic unhealthy container restart (optional)
- Email alerts on issues

Examples:

```bash
# Start dashboard (60s refresh)
sudo bash vps-docker-health.sh

# Faster updates (30s)
sudo bash vps-docker-health.sh --interval=30

# Auto-restart unhealthy containers
sudo bash vps-docker-health.sh --restart-unhealthy

# Email alerts
sudo bash vps-docker-health.sh --alert-email=admin@example.com
```

---

vps-docker-log-rotation.sh

Configure Docker container log rotation and size limits.

Usage:

```bash
sudo bash vps-docker-log-rotation.sh [--max-size=100m] [--max-file=5] [--apply] [--alert-email=email]
```

Configuration:

Update `/etc/docker/daemon.json`:

```json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m",
    "max-file": "5"
  }
}
```

Log Drivers:

- `json-file`: Default, supports rotation (current example)
- `local`: Better performance, built-in rotation
- Other: syslog, journald, splunk, awslogs

Features:

- Check current log sizes
- Logging driver configuration
- Daemon-wide log settings
- Per-container configuration
- Log truncation
- Automatic cleanup of old logs

Examples:

```bash
# Check log sizes
sudo bash vps-docker-log-rotation.sh

# Apply rotation settings
sudo bash vps-docker-log-rotation.sh --max-size=100m --max-file=5 --apply

# Larger logs for busy apps
sudo bash vps-docker-log-rotation.sh --max-size=500m --max-file=10 --apply
```

---

vps-docker-cleanup.sh

Remove unused Docker images, volumes, and networks.

Usage:

```bash
sudo bash vps-docker-cleanup.sh [--dry-run] [--aggressive] [--prune-all]
```

Cleanup Operations:

Option	Action
(default)	Dangling images + unused volumes
--aggressive	+ stopped containers
--prune-all	+ system prune (all untagged)

Features:

- Dangling image removal
- Unused image detection
- Unused volume cleanup
- Orphaned network removal
- Build cache pruning
- Old log cleanup
- Disk usage reporting

Examples:

```bash
# Test first with dry-run
sudo bash vps-docker-cleanup.sh --dry-run

# Remove dangling and unused
sudo bash vps-docker-cleanup.sh

# Also remove stopped containers
sudo bash vps-docker-cleanup.sh --aggressive

# Complete cleanup
sudo bash vps-docker-cleanup.sh --aggressive --prune-all
```

---

vps-docker-backup-restore.sh

Automated container and volume backup/restore.

Usage:

```bash
sudo bash vps-docker-backup-restore.sh --mode=backup|restore|status [options]
```

Modes:

Backup:

```bash
# All containers and volumes
sudo bash vps-docker-backup-restore.sh --mode=backup --containers=all --backup-dir=/opt/backups

# Specific container
sudo bash vps-docker-backup-restore.sh --mode=backup --containers=coolify

# Volumes only
sudo bash vps-docker-backup-restore.sh --mode=backup --volumes=all
```

Restore:

```bash
# From backup file
sudo bash vps-docker-backup-restore.sh --mode=restore --backup-file=/opt/backups/full-backup.tar.gz
```

Status:

```bash
# View backup inventory
sudo bash vps-docker-backup-restore.sh --mode=status
```

Features:

- Full container+volume backup
- Individual container backup
- Config/filesystem/volume separation
- Manifest generation
- Automatic compression

- Retention policies
- Backup verification

Maintenance

vps-automated-cleanup.sh

System-wide cleanup of logs, temp files, and cache.

Usage:

```bash
sudo bash vps-automated-cleanup.sh [--dry-run] [--aggressive]
```

Cleanup Operations:

Operation	Default	--aggressive
Rotated logs (>30d)	Yes	Yes
Journal logs (>2w)	Yes	Yes
/tmp files (>7d)	Yes	Yes
/var/tmp files (>7d)	Yes	Yes
APT cache	Yes	Yes
Broken symlinks	Yes	Yes
Core dumps	No	Yes
Old backups (>90d)	No	Yes
Disk TRIM	No	Yes

Examples:

```bash
```

```
# Test with dry-run
sudo bash vps-automated-cleanup.sh --dry-run

# Run standard cleanup
sudo bash vps-automated-cleanup.sh

# Full cleanup including backups
sudo bash vps-automated-cleanup.sh --aggressive
```

---

vps-package-updater.sh

Manage system and application package updates.

Usage:

```bash
sudo bash vps-package-updater.sh [--check|--update-system|--update-docker] [--alert-email=email]
```

Checks:

- Security updates (critical)
- Regular updates
- Docker version status
- Container image updates
- Python package status
- Kernel updates

Examples:

```bash
```

```bash
# Check available updates
sudo bash vps-package-updater.sh --check

# Update system packages
sudo bash vps-package-updater.sh --update-system

# Update Docker and restart containers
sudo bash vps-package-updater.sh --update-docker

# Alert on security updates
sudo bash vps-package-updater.sh --check --alert-email=admin@example.com
```

---

vps-database-backup.sh

Backup PostgreSQL, MySQL, and MongoDB databases.

Usage:

```bash
sudo bash vps-database-backup.sh [--type=all|postgresql|mysql|mongodb] [--backup-dir=/opt/backups] [--retention=30]
```

Backup Methods:

Database	Method
PostgreSQL	pg_dump per database + globals
MySQL	mysqldump all-databases + individual
MongoDB	mongodump compression

Features:

- Multi-database backup
- Automatic compression
- Integrity verification
- Retention policies
- Old backup cleanup
- Database accessibility checks

Examples:

```bash
# Backup all databases
sudo bash vps-database-backup.sh --type=all

# PostgreSQL only
sudo bash vps-database-backup.sh --type=postgresql

# Custom retention (90 days)
sudo bash vps-database-backup.sh --type=all --retention=90

# Verify backups
sudo bash vps-database-backup.sh --type=all --verify-all
```

---

vps-system-upgrade.sh

Safe major OS upgrades with validation and rollback.

Usage:

```bash
sudo bash vps-system-upgrade.sh [--dry-run] [--backup] [--skip-docker-pull]
```

Process:

1. **Pre-flight checks**
   - Disk space validation (min 1GB)
   - Service availability
   - SSH connectivity
2. **Backup**
   - Essential configs backup
   - Container state commits
   - Pre-upgrade snapshot
3. **Upgrade**
   - Service shutdown
   - Package upgrades
   - Kernel updates

4. **Restart & Verify**
    - Service restart
    - Health verification
    - Reboot notification if needed

Examples:

```bash
# Test without changes
sudo bash vps-system-upgrade.sh --dry-run

# Full upgrade with backup
sudo bash vps-system-upgrade.sh --backup

# Skip docker image pulling
sudo bash vps-system-upgrade.sh --backup --skip-docker-pull
```

Orchestration

vps-orchestration.sh

Master control script combining all monitoring and maintenance tasks.

Usage:

```bash
sudo bash vps-orchestration.sh [--mode=report|monitor|maintain|full] [--email=admin@example.com]
```

Modes:

Mode	Tasks
report	Monitoring + security audit + summary (default)
monitor	All monitoring checks only
maintain	Maintenance tasks only
full	Everything: monitoring + maintenance + cleanup

Examples:

```bash
# Generate report
sudo bash vps-orchestration.sh --mode=report

# Full suite with email
sudo bash vps-orchestration.sh --mode=full --email=admin@example.com

# Monitoring only
sudo bash vps-orchestration.sh --mode=monitor
```

Configuration

Global Configuration Directory

```bash
sudo mkdir -p /etc/vps-tools
```

Cron Installation

```bash
# Copy cron config
sudo cp vps-tools-cron.conf /etc/cron.d/vps-tools

# Edit email address
sudo nano /etc/cron.d/vps-tools

# Verify
sudo systemctl restart cron
```

Custom Alert Email

Set per-script:

```bash
--alert-email=your-email@domain.com
```

Or globally in `/etc/cron.d/vps-tools`:

```bash
```

```
MAILTO=your-email@domain.com
```

Troubleshooting

Scripts Return "Docker not installed"

```bash
# Install Docker
sudo bash vps-build.sh  # Select Docker + Portainer
# Or manually:
sudo apt-get update
sudo apt-get install -y docker.io
```

Permission Denied Errors

All scripts require root:

```bash
# Correct
sudo bash script-name.sh

# Incorrect
bash script-name.sh
```

Email Alerts Not Sending

```bash
# Install mailutils
sudo apt-get install mailutils

# Test mail
echo "Test" | mail -s "Test Subject" your-email@domain.com
```

Cron Jobs Not Executing

```bash
```

```bash
# Check cron daemon
sudo systemctl status cron

# View cron logs
sudo journalctl -u cron --follow

# Verify permissions
ls -la /etc/cron.d/vps-tools
```

Out of Memory During Backups

```bash
# Reduce backup size or increase swap
sudo bash vps-build.sh  # Reconfigure: increase swap

# Or limit backup scope
--containers=specific-container
```

SSH Key Rotation Fails

```bash
# Verify .ssh permissions
ls -la ~/.ssh
# Should be: drwx------ (700)

# Fix if needed
chmod 700 ~/.ssh
chmod 600 ~/.ssh/*
```

Common Task Combinations

Daily System Check

```bash
sudo bash monitoring/vps-health-monitor.sh
sudo bash docker/vps-docker-health.sh --interval=1
sudo bash security/vps-open-ports-auditor.sh
```

Weekly Security Audit

```bash
sudo bash security/vps-ssh-audit.sh --audit
sudo bash security/vps-failed-login-reporter.sh --days=7
sudo bash security/vps-ssl-checker.sh
```

Monthly Maintenance

```bash
sudo bash maintenance/vps-package-updater.sh --check
sudo bash maintenance/vps-database-backup.sh --type=all
sudo bash maintenance/vps-automated-cleanup.sh --aggressive
sudo bash docker/vps-docker-backup-restore.sh --mode=backup --containers=all
```

Pre-Upgrade Process

```bash
sudo bash maintenance/vps-system-upgrade.sh --dry-run
sudo bash docker/vps-docker-backup-restore.sh --mode=backup --containers=all
sudo bash maintenance/vps-system-upgrade.sh --backup
```

Performance Considerations

Resource Usage

Script	CPU	Memory	Disk I/O	Duration
health-monitor	Low	Low	Low	<5s
service-monitor	Low	Low	Low	<5s
log-analyzer	Medium	Low	Medium	10-30s
ssh-audit	Low	Low	Low	<5s
docker-backup	High	Medium	High	Minutes
system-upgrade	High	Medium	High	Minutes

Optimization Tips

- Run backups during off-peak hours
- Use --dry-run before major operations
- Stagger cron jobs to avoid concurrent runs
- Monitor system load during scheduled tasks
- Archive old logs to manage disk space
- Use aggressive cleanup judiciously

Support & Updates

For issues or enhancements:

1. Check script logs: `/var/log/vps-tools/`
2. Run health checks: `sudo bash monitoring/vps-health-monitor.sh`
3. Test with --dry-run before production use
4. Review output colors and status messages
5. Check script version: `head -5 script-name.sh`